

ShopZone

Backend Developer Documentation

E-Commerce Platform — React + Node.js + MongoDB

Melvin Waweru Munyua | github.com/Mellvvin | 2025

This document covers every file built in the backend phase of ShopZone, including all terminal commands used, what every function does line by line, key concepts explained simply, and a complete reference for continuing development.

CONTENTS

1. Project Overview & Tech Stack
2. Environment Setup — All Commands
3. Project Structure
4. File 1 — .env (Environment Variables)
5. File 2 — .gitignore
6. File 3 — server.js (The Server)
7. File 4 — models/User.js
8. File 5 — models/Product.js
9. File 6 — models/Order.js
10. File 7 — middleware/authMiddleware.js
11. File 8 — controllers/userController.js
12. File 9 — controllers/productController.js
13. File 10 — controllers/orderController.js
14. Key Concepts Reference
15. HTTP Status Codes
16. What Comes Next

1. PROJECT OVERVIEW & TECH STACK

ShopZone is a full-stack e-commerce web application. Customers can browse products, register accounts, log in, add items to a cart, place orders, and pay. Admins can add, edit, and delete products and manage orders.

Tech stack

Layer	Technology	What it does
Frontend	React.js	The user interface — what customers see in the browser
Backend	Node.js + Express	The server — handles all requests

		and business logic
Database	MongoDB + Mongoose	Stores all users, products, and orders
Authentication	JWT + bcryptjs	Secure login tokens and password encryption
Dev Tool	Nodemon	Auto-restarts the server when you save a file
Config	dotenv	Loads secret values from the .env file
Security	cors	Allows the React frontend to talk to the Node backend

2. ENVIRONMENT SETUP — ALL COMMANDS

These are every terminal command run from the very beginning of the project in the exact order they were executed.

Fix PowerShell script execution policy

Windows blocks scripts by default. This command allows locally written scripts and trusted scripts to run for your user account only.

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Navigate to Desktop and create project folder

```
cd C:\Users\user\OneDrive\Desktop
mkdir ecommerce-platform
cd ecommerce-platform
mkdir backend
mkdir frontend
cd backend
```

Initialise the Node.js project

Creates package.json — the ID card of the project that records all dependencies.

```
npm init -y
```

Install all backend dependencies

Downloads and installs the 6 packages the backend needs to work.

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
```

Install nodemon as a development tool

Nodemon watches your files and automatically restarts the server every time you save. The --save-dev flag marks it as a development-only tool.

```
npm install --save-dev nodemon
```

Open the full project in VS Code

```
cd ..  
code .
```

Create all files and folders

```
cd backend  
New-Item .env  
New-Item .gitignore  
New-Item server.js  
mkdir models  
mkdir routes  
mkdir middleware  
mkdir controllers  
New-Item models/User.js  
New-Item models/Product.js  
New-Item models/Order.js  
New-Item middleware/authMiddleware.js  
New-Item controllers/userController.js  
New-Item controllers/productController.js  
New-Item controllers/orderController.js
```

Start the development server

Use this every time you come back to work on the project.

```
cd C:\Users\user\OneDrive\Desktop\ecommerce-platform\backend  
npm run dev
```

IMPORTANT: Always open a **SECOND** terminal (Ctrl + Shift + `) for running commands. Keep nodemon running in the first terminal at all times. Never close the first terminal while working.

3. PROJECT STRUCTURE

This is the complete folder and file structure of the backend at the end of this phase.

```
ecommerce-platform/  
├── backend/  
│   ├── controllers/  
│   │   ├── userController.js  
│   │   ├── productController.js  
│   │   └── orderController.js  
│   ├── middleware/  
│   │   └── authMiddleware.js  
│   ├── models/  
│   │   ├── User.js  
│   │   ├── Product.js  
│   │   └── Order.js  
│   ├── node_modules/      ← auto-generated, never edit  
│   ├── routes/           ← empty, built in next session  
│   ├── .env              ← secret config, never on GitHub  
│   ├── .gitignore  
│   └── package.json
```

```

├── package-lock.json    ← auto-generated, never edit
├── server.js
└── frontend/           ← React app, built after backend

```

What each folder is responsible for

Folder	Responsibility
models/	Defines the shape of data stored in MongoDB — what fields each document has
controllers/	Contains the actual logic — what happens when a request comes in
middleware/	Code that runs between a request and a response — used for authentication
routes/	Defines the API URLs and connects them to the right controller functions

4. FILE 1 — .ENV (ENVIRONMENT VARIABLES)

The .env file stores secret configuration values. It is never uploaded to GitHub because it contains sensitive information like database passwords and secret keys.

```

PORT=5000
MONGO_URI=mongodb://localhost:27017/shopzone
JWT_SECRET=melvin_shopzone_secret_key_2025

```

What each line does

- PORT=5000 — tells the Express server which port to run on. Port 5000 means you access the API at `http://localhost:5000`
- MONGO_URI — the full address of the MongoDB database. localhost means the database is on your own computer. 27017 is the default MongoDB port. shopzone is the name of the database — MongoDB creates it automatically the first time data is saved
- JWT_SECRET — a private key used to sign and verify login tokens. Think of it like a stamp that only your server knows. Anyone without this key cannot create valid tokens

When deploying ShopZone to the internet, you change MONGO_URI to point to a cloud database like MongoDB Atlas and update JWT_SECRET to a stronger random string. The code in server.js never needs to change — only the .env values.

5. FILE 2 — .GITIGNORE

The .gitignore file tells Git which files and folders to ignore and never upload to GitHub.

```

node_modules
.env

```

- node_modules — this folder contains all 100 downloaded packages and is very large. Anyone who clones your project just runs npm install to recreate it. No need to upload it
- .env — your secret keys must never be on GitHub. If someone finds your JWT_SECRET they can forge login tokens and access any account

6. FILE 3 — SERVER.JS (THE SERVER)

This is the entry point of the entire backend. It is the first file Node.js reads when you run npm run dev. It sets up Express, connects to MongoDB, and starts listening for incoming requests.

```
const express    = require('express');
const mongoose   = require('mongoose');
const cors       = require('cors');
const dotenv     = require('dotenv');

dotenv.config();

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected successfully'))
  .catch((err) => console.log('MongoDB connection error:', err));

app.get('/', (req, res) => {
  res.send('ShopZone API is running');
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Line by line explanation

- require('express'), require('mongoose') etc. — imports the installed packages. Like telling JavaScript 'go get these tools from the node_modules folder'
- dotenv.config() — reads the .env file and loads PORT, MONGO_URI and JWT_SECRET into process.env so the rest of the code can access them
- const app = express() — creates the Express application. Think of it like building the shell of a restaurant — the structure exists but nothing is happening yet
- app.use(cors()) — allows the React frontend (running on port 3000) to send requests to this server (running on port 5000). Without this the browser blocks cross-origin requests
- app.use(express.json()) — tells Express to automatically parse JSON data sent in request bodies. Without this req.body would be undefined
- mongoose.connect(process.env.MONGO_URI) — connects to MongoDB using the URI from .env. The .then prints success, the .catch prints any connection error
- app.get('/') — defines the root route. When someone visits http://localhost:5000 in the browser they see 'ShopZone API is running'. This is how we know the server is working
- app.listen(PORT) — this is the line that actually STARTS the server. It tells the computer to keep running and listen on port 5000 for incoming requests. This is what creates the server

package.json scripts section

After installing nodemon we added two scripts to package.json:

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "start": "node server.js",
  "dev": "nodemon server.js"
},
```

- npm start — runs the server with plain Node.js. Used in production
- npm run dev — runs the server with nodemon. Used during development. Auto-restarts on every file save

7. FILE 4 — MODELS/USER.JS

Defines the structure (schema) of every user stored in MongoDB. Think of it like designing a registration form — it defines exactly what fields a user must have, what type of data each field holds, and any validation rules.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true,
    lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6 },
  isAdmin: { type: Boolean, default: false },
}, { timestamps: true });

const User = mongoose.model('User', userSchema);
module.exports = User;
```

Field by field explanation

- name — String type. required: true means every user must have a name. trim: true automatically removes leading and trailing spaces
- email — String type. unique: true ensures no two users can register with the same email. lowercase: true converts all emails to lowercase so Melvin@gmail.com and melvin@gmail.com are treated as the same address
- password — String type. minlength: 6 rejects passwords shorter than 6 characters. The actual password is never stored — bcrypt encrypts it before saving
- isAdmin — Boolean (true/false). Every new user automatically gets false. Only the shop owner gets true. This controls who can add/edit/delete products
- timestamps: true — MongoDB automatically adds createdAt and updatedAt fields to every user document. Useful for knowing when someone signed up

What mongoose.model does

mongoose.model('User', userSchema) creates the actual model. MongoDB stores all users in a collection called 'users' (automatically made lowercase and plural). module.exports = User makes this model available to any other file that needs it.

Key concept — Schema vs Model vs Collection: Schema = the blueprint (what fields exist). Model = the builder tool that creates/reads/updates/deletes documents. Collection = where documents are actually stored in MongoDB (equivalent to a table in SQL).

8. FILE 5 — MODELS/PRODUCT.JS

Defines the structure of every product in the store. This file contains two schemas — a reviewSchema for individual reviews and a productSchema for products. A product embeds multiple reviews directly inside it.

reviewSchema — the nested review structure

```
const reviewSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId,
    required: true, ref: 'User' },
  name: { type: String, required: true },
  rating: { type: Number, required: true, min: 1, max: 5 },
  comment: { type: String, required: true },
}, { timestamps: true });
```

- user — stores the ObjectId of the user who wrote the review. ref: 'User' links it to the User collection
- rating — Number type with min: 1 and max: 5. MongoDB validates this automatically and rejects anything outside that range

productSchema — the main product structure

```
const productSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId,
    required: true, ref: 'User' },
  name: { type: String, required: true, trim: true },
  image: { type: String, required: true },
  category: { type: String, required: true, trim: true },
  description: { type: String, required: true },
  reviews: [reviewSchema],
  rating: { type: Number, required: true, default: 0 },
  numReviews: { type: Number, required: true, default: 0 },
  price: { type: Number, required: true, default: 0 },
  countInStock: { type: Number, required: true, default: 0 },
}, { timestamps: true });
```

- user — the admin who created this product. Linked to User collection via ObjectId
- image — stores the URL path to the product image e.g. /images/nike-air-max.jpg
- reviews: [reviewSchema] — the square brackets mean this is an array. A product can have zero or many reviews. Each review follows the reviewSchema defined above
- rating — the average star rating. Starts at 0. Automatically recalculated in the product controller every time a new review is added
- numReviews — total number of reviews. Starts at 0. Incremented in the controller when a review is added
- countInStock — how many units are available. When someone buys one this number decreases

ObjectId and ref — these are how MongoDB links two collections together. When we write ref: 'User' we are saying this field stores the _id of a document in the User collection. Later we use .populate() to replace that ID with the full user object when fetching data.

9. FILE 6 — MODELS/ORDER.JS

Defines the structure of every order placed on ShopZone. Like Product.js it has two schemas — an orderItemSchema for individual cart items and an orderSchema for the complete order.

orderItemSchema

```
const orderItemSchema = new mongoose.Schema({
  name:    { type: String, required: true },
  qty:     { type: Number, required: true },
  image:   { type: String, required: true },
  price:   { type: Number, required: true },
  product: { type: mongoose.Schema.Types.ObjectId,
             required: true, ref: 'Product' },
});
```

- product — links each order item back to the actual product in the Product collection via ObjectId
- qty — the quantity ordered. If a customer orders 3 of the same product qty is 3

orderSchema key fields

- user — links the order to the customer who placed it
- orderItems: [orderItemSchema] — array of all items purchased in this order
- shippingAddress — a nested object with address, city, postalCode, and country fields
- paymentMethod — e.g. M-Pesa, PayPal, Credit Card
- paymentResult — stores the response from the payment provider: transaction ID, status, timestamp, and email
- itemsPrice, shippingPrice, taxPrice, totalPrice — the price breakdown. $\text{totalPrice} = \text{itemsPrice} + \text{shippingPrice} + \text{taxPrice}$
- isPaid — Boolean, starts false. Set to true when payment is confirmed
- paidAt — Date field. Empty until payment happens, then records the exact payment timestamp
- isDelivered — Boolean, starts false. Admin sets it to true when order ships
- deliveredAt — Date field. Empty until admin marks as delivered

How all three models connect

```
User
├── places an Order
│   ├── contains OrderItems
│   │   └── each references a Product
```

User also creates Products (admin only)
User also writes Reviews (embedded in Product)

10. FILE 7 — MIDDLEWARE/AUTHMIDDLEWARE.JS

Middleware is code that runs between a request arriving and a response being sent. This file acts as a security guard — it checks every protected request to confirm the user is genuinely logged in before allowing them through.

```
const jwt = require('jsonwebtoken');
```



```

const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      next();
    } catch (error) {
      res.status(401).json({ message: 'Not authorised, token failed' });
    }
  }
  if (!token) {
    res.status(401).json({ message: 'Not authorised, no token' });
  }
};

const admin = (req, res, next) => {
  if (req.user && req.user.isAdmin) {
    next();
  } else {
    res.status(401).json({ message: 'Not authorised as admin' });
  }
};

module.exports = { protect, admin };

```

protect function — line by line

- req.headers.authorization — when a user logs in we give them a token. Every request after that must include this token in the Authorization header
- startsWith('Bearer') — tokens are sent as 'Bearer abc123...'. We check it starts with the word Bearer to confirm the format is correct
- split(' ')[1] — splits 'Bearer abc123...' by the space and takes the second part — just the token itself without the word Bearer
- jwt.verify(token, process.env.JWT_SECRET) — checks the token is valid and has not been tampered with using our secret key from .env
- decoded.id — the token contains the user's ID encoded inside it. After verifying we extract that ID
- User.findById(decoded.id).select('-password') — finds the user in MongoDB using that ID. The minus sign in .select('-password') means give me everything EXCEPT the password field
- req.user = ... — attaches the full user object to the request so the next function knows exactly who is making the request
- next() — means this user is verified and trusted. Let them through to the actual route handler

admin function — line by line

- Runs AFTER protect. By this point req.user is already set
- Checks if req.user.isAdmin is true — if yes, calls next() and lets them through
- If isAdmin is false, sends back 401 (Unauthorised). Even logged-in users cannot access admin routes

How middleware is used in routes (preview)

```
// Any logged in user can access this:
router.get('/profile', protect, getUserProfile);

// Only admin users can access this:
router.delete('/product/:id', protect, admin, deleteProduct);
```

protect and admin run in order from left to right before the actual controller function runs.

11. FILE 8 — CONTROLLERS/USERCONTROLLER.JS

Contains all the logic for user-related operations. When a request hits a user route this is the code that runs. It exports four functions: registerUser, loginUser, getUserProfile, and updateUserProfile.

generateToken — helper function

```
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: '30d',
  });
};
```

- `jwt.sign({ id }, ...)` — creates a JWT token with the user's ID encoded inside it
- `process.env.JWT_SECRET` — uses the secret key from `.env` to sign the token. Anyone without this key cannot create or read a valid token
- `expiresIn: '30d'` — the token expires after 30 days. After that the user must log in again
- This function is used at the end of both `registerUser` and `loginUser` to give the user their token

registerUser — handles new account creation

```
const registerUser = async (req, res) => {
  const { name, email, password } = req.body;
  try {
    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: 'User already exists' });
    }
    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    const user = await User.create({
      name, email, password: hashedPassword,
    });
    if (user) {
      res.status(201).json({
        _id: user._id, name: user.name,
        email: user.email, isAdmin: user.isAdmin,
        token: generateToken(user._id),
      });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

- `const { name, email, password } = req.body` — destructuring. Pulls all three values out of the request body in one line instead of writing `req.body.name`, `req.body.email` etc.

- `User.findOne({ email })` — searches MongoDB for an existing user with that email. If found we reject the registration immediately
- `bcrypt.genSalt(10)` — generates a 'salt' — random data that gets added to the password before hashing. The 10 is the cost factor — how many rounds of encryption to apply. Higher = more secure but slower. 10 is the industry standard
- `bcrypt.hash(password, salt)` — combines the plain text password with the salt and creates a hash. The result looks like `$2b$10$xK8...` and can never be reversed back to the original password
- `User.create({...})` — saves the new user to MongoDB with the hashed password. The plain password is never stored anywhere
- `res.status(201).json({...})` — sends back user details plus a token. 201 means 'Created'. The user is automatically logged in after registering

loginUser — handles authentication

```
const loginUser = async (req, res) => {
  const { email, password } = req.body;
  try {
    const user = await User.findOne({ email });
    if (user && (await bcrypt.compare(password, user.password))) {
      res.json({
        _id: user._id, name: user.name,
        email: user.email, isAdmin: user.isAdmin,
        token: generateToken(user._id),
      });
    } else {
      res.status(401).json({ message: 'Invalid email or password' });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

- `User.findOne({ email })` — finds the user by email first
- `bcrypt.compare(password, user.password)` — compares the plain text password the user just typed with the hashed version stored in the database. Bcrypt knows how to do this comparison without unhashing. Returns true or false
- If both the user exists AND the password matches: send back user data and a fresh token
- If either fails: send 401 (Unauthorised). We deliberately use a vague message 'Invalid email or password' — never tell an attacker which one was wrong

getUserProfile — fetch logged in user's details

- Uses `req.user._id` — the protect middleware already attached the user to the request. So we already know exactly who is asking
- Finds the user in MongoDB by that ID and sends back their details (without password)
- This route is protected — only accessible with a valid token

updateUserProfile — let user update their details

- `user.name = req.body.name || user.name` — if a new name was sent in the request use it, otherwise keep the existing name. The `||` (OR) operator is the key here
- Only re-hashes the password if `req.body.password` was actually sent — no need to rehash if they did not change it
- Saves the updated user with `user.save()` and returns fresh data with a new token

12. FILE 9 — CONTROLLERS/PRODUCTCONTROLLER.JS

Contains all the logic for product-related operations. Exports six functions: `getProducts`, `getProductById`, `createProduct`, `updateProduct`, `deleteProduct`, and `createProductReview`.

`getProducts` — fetch all products

```
const getProducts = async (req, res) => {
  try {
    const products = await Product.find({});
    res.json(products);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

- `Product.find({})` — the empty curly braces mean no filter. Find ALL documents in the products collection
- Sends back an array of every product to the frontend

`getProductById` — fetch one product

- `req.params.id` — extracts the ID from the URL. When someone requests `/api/products/64f1a2b3`, that `64f1a2b3` is `req.params.id`
- `Product.findById(req.params.id)` — finds the specific product with that ID
- If found send it back. If not send 404 (Not Found)

`createProduct` — admin adds a new product

- Uses `req.user._id` from the protect middleware to record which admin created the product
- `new Product({...})` creates the product in memory. `product.save()` then saves it to MongoDB
- Returns 201 (Created) with the new product data
- This route will be protected with both protect AND admin middleware in the routes file

`updateProduct` — admin edits a product

- Finds the product by ID first to confirm it exists
- Uses the same `||` pattern as `updateUserProfile` — only updates fields that were actually sent in the request
- `product.save()` saves all changes back to MongoDB

`deleteProduct` — admin removes a product

- Finds the product by ID first
- `product.deleteOne()` permanently removes it from the database
- Sends back a success message confirming deletion

`createProductReview` — customer adds a review

```
const alreadyReviewed = product.reviews.find(
  (r) => r.user.toString() === req.user._id.toString()
);

product.reviews.push(review);
product.numReviews = product.reviews.length;
```

```
product.rating =
  product.reviews.reduce((acc, item) => item.rating + acc, 0) /
  product.reviews.length;
```

- alreadyReviewed — loops through all existing reviews and checks if this user already left one. One review per user per product
- product.reviews.push(review) — adds the new review to the reviews array
- product.numReviews = product.reviews.length — counts the total reviews after adding the new one
- The reduce() call adds up all ratings then divides by the count to get the new average rating. Example: ratings of 5, 4, 3 → (5+4+3) / 3 = 4.0
- This is the rating recalculation logic that was 'coming later in the controller' when we first defined rating: 0 in the Product model

13. FILE 10 — CONTROLLERS/ORDERCONTROLLER.JS

Contains all the logic for order-related operations. Exports six functions: createOrder, getOrderById, updateOrderToPaid, updateOrderToDelivered, getMyOrders, and getAllOrders.

createOrder — customer places an order

- Destructures all order details from req.body in one line: orderItems, shippingAddress, paymentMethod, and all price fields
- orderItems.length === 0 — checks if the cart is empty. Rejects empty orders immediately with a 400 error
- new Order({...}) — creates the order object in memory. user: req.user._id records who placed it
- order.save() — saves to MongoDB and returns the created order with status 201

getOrderById — fetch one order

- Order.findById(req.params.id) — finds the order by its ID from the URL
- .populate('user', 'name email') — replaces the user ObjectId with the actual user's name and email from the User collection. The second argument limits which fields are populated
- This is populate() in action — the concept introduced when we defined ref: 'User' in the models

updateOrderToPaid — confirm payment

- order.isPaid = true — flips the paid status
- order.paidAt = Date.now() — records the exact timestamp of payment
- order.paymentResult = {...} — stores the full response from the payment provider (PayPal, M-Pesa etc.) including transaction ID, status, time and email
- order.save() — commits all changes to MongoDB

updateOrderToDelivered — admin marks order shipped

- order.isDelivered = true — flips the delivered status
- order.deliveredAt = Date.now() — records exact delivery timestamp
- Only admins can call this — protected by both protect and admin middleware in the routes

getMyOrders — customer sees their own orders

- Order.find({ user: req.user._id }) — finds ONLY orders where the user field matches the logged in user's ID
- Melvin only sees Melvin's orders. No one else's orders are ever returned

getAllOrders — admin sees every order

- Order.find({}) — no filter. Returns every single order in the database
- .populate('user', 'id name') — replaces each user ObjectId with the user's id and name so the admin can see who placed each order
- Only accessible to admins via the admin middleware

14. KEY CONCEPTS REFERENCE

ObjectId

MongoDB's unique identifier automatically assigned to every document. It is a 24-character hexadecimal string (e.g. 64f1a2b3c4d5e6f7a8b9c0d1) built from a timestamp, machine ID, and random counter. No two documents anywhere in the world will have the same ObjectId.

Schema vs Model vs Collection

Term	What it is	Analogy
Schema	The blueprint — defines fields and rules	A blank form template
Model	The tool to create/read/update/delete documents	The form processor
Collection	Where documents are actually stored in MongoDB	A filing cabinet drawer
Document	One individual record in a collection	One filled-in form

JWT (JSON Web Token)

A token given to a user after they log in. It contains the user's ID encoded inside it and is signed with a secret key. Every protected request must include this token. The server verifies it using the same secret key. Tokens expire after 30 days. Format: Bearer eyJhbGci...

bcrypt

A one-way encryption library for passwords. Hashing is irreversible — you cannot get the original password back from the hash. When a user logs in bcrypt compares the typed password against the stored hash without ever decrypting it.

populate()

A Mongoose feature that replaces an ObjectId reference with the actual document it points to. Example: product.user is stored as an ObjectId. After .populate('user') it becomes the full user object with name, email etc.

async/await

Database operations take time. `async` marks a function as asynchronous. `await` pauses execution until the database responds before moving to the next line. The `try/catch` around every `await` catches any errors and sends a 500 response instead of crashing the server.

Middleware chain

Express processes routes left to right. In `router.get('/profile', protect, getUserProfile)` — `protect` runs first. If it calls `next()` then `getUserProfile` runs. If `protect` sends a response (like 401) `getUserProfile` never runs. This is how authentication works.

15. HTTP STATUS CODES USED

Code	Meaning	When we use it
200	OK	Default success — data sent back
201	Created	Something new was saved to the database
400	Bad Request	Data sent was wrong, missing, or invalid
401	Unauthorised	Not logged in, wrong token, or not an admin
404	Not Found	The item does not exist in the database
500	Server Error	Something unexpected broke on the server side

16. WHAT COMES NEXT

The backend logic is complete. The next session will build the routes folder and then test everything with Postman before moving to the React frontend.

Files remaining in the backend

File	Location	What it will do
<code>userRoutes.js</code>	<code>routes/userRoutes.js</code>	Define URLs for register, login, profile
<code>productRoutes.js</code>	<code>routes/productRoutes.js</code>	Define URLs for all product operations
<code>orderRoutes.js</code>	<code>routes/orderRoutes.js</code>	Define URLs for all order operations

How to pick up in a new chat session

Paste this at the start of your next chat: "I am building a full stack e-commerce app called ShopZone with React, Node.js and MongoDB. We have completed the backend models (`User.js`, `Product.js`, `Order.js`), middleware (`authMiddleware.js`) and all three controllers (`UserController.js`,

productController.js, orderController.js). We are now on File 11 of 15 — starting the routes folder with userRoutes.js. My project is at C:\Users\user\OneDrive\Desktop\ecommerce-platform. Please continue from where we left off."

How to restart your dev server each session

```
cd C:\Users\user\OneDrive\Desktop\ecommerce-platform\backend
npm run dev

// Then open second terminal with Ctrl + Shift + `
cd C:\Users\user\OneDrive\Desktop\ecommerce-platform\backend
```

Full project roadmap

Phase	What we build	Status
Backend — Models	User.js, Product.js, Order.js	Complete
Backend — Middleware	authMiddleware.js	Complete
Backend — Controllers	userController, productController, orderController	Complete
Backend — Routes	userRoutes, productRoutes, orderRoutes	Next session
Backend — Testing	Test all endpoints with Postman	Next session
Frontend — Setup	Create React app, install dependencies	Coming soon
Frontend — Auth	Login and Register pages	Coming soon
Frontend — Products	Product listing, detail, and admin pages	Coming soon
Frontend — Cart	Cart and checkout flow	Coming soon
GitHub	Push full project, write README	Final step

ShopZone Backend Documentation | Melvin Waweru Munyua | github.com/Mellvvin